

Selenium – Overview

Selenium is an open-source automation testing tool primarily used for automating **web applications** across different browsers and platforms. It supports multiple programming languages such as **Java, Python, C#, JavaScript, and Ruby**, making it flexible for testers and developers.

Selenium works by interacting with web elements the same way a real user would (clicking buttons, entering text, navigating pages). It is widely used in the industry because it is **free, browser-independent**, and easily **integrated with frameworks, CI/CD tools, and build tools**.

1. Automation Tools

Commonly used automation testing tools include: - **Selenium** – Open-source tool for automating web applications. - **Cypress** – JavaScript-based end-to-end testing framework mainly for modern web apps. - **Katalon Studio** – Built on top of Selenium and Appium with built-in features for easier automation. - **Protractor** – Automation tool mainly used for Angular applications (now deprecated but still referenced).

2. Selenium – Limitations

Selenium has the following limitations: - Cannot automate **desktop applications** (e.g., Calendar, Taskbar, File Explorer). - Cannot automate: - Pie charts - Bar charts / graphs - OTP (One-Time Passwords) - Barcodes / QR codes

3. Uses of Selenium

Selenium is used for: - Functional testing - Smoke testing scenarios - Regression testing - End-to-End testing

4. Components of Selenium

4.1 Selenium IDE

- Integrated Development Environment (browser plugin).
- Used for **record and playback** of test cases.
- Suitable only for basic testing (not complex logic).

4.2 Selenium RC (Remote Control)

- Acts as an **intermediate server**.
- Proxies commands from Selenium IDE to the browser.
- Slower compared to WebDriver.

4.3 Selenium WebDriver

- A framework that **directly communicates with the browser** (no RC required).
- Faster and more reliable.
- Supports multiple programming languages (Java, Python, C#, etc.).

4.4 Selenium Grid

- Used for **parallel execution** of test cases.
- Supports execution on multiple browsers and machines.

Note: Selenium 2 = Selenium RC + Selenium WebDriver

5. Important Points

- To automate **Firefox**, `geckodriver` is required because Firefox does not support legacy direct connections.
 - Selenium has a built-in tool called **Selenium Manager** which automatically handles driver setup (mainly for Firefox).
-

6. Selenium Integration with Build Tools

Selenium can be integrated with: - **Maven** - **Ant**

These tools: - Manage project dependencies - Compile code - Execute test cases as part of the build process

7. Selenium IDE – Working

Steps: 1. Install Selenium IDE browser plugin. 2. Provide the **base URL** before recording. 3. Start recording actions.

Each recorded step has: - **Command** – Action performed (click, type, set window size, etc.) - **Target** – Web element locator (id, name, xpath, etc.) - **Value** – Input value entered

Features: - Delete or reorder commands. - Right-click elements to add **verify / assert** steps. - Target dropdown provides multiple locator options. - Execution log shows pass/fail status. - `verify` failure → test continues with error message. - `assert` failure → test stops immediately.

8. Selenium with Java – Setup

Steps: 1. Install **JDK**. 2. Download and install **Eclipse IDE**. 3. Download Selenium Java libraries from selenium.dev. 4. Create a Java project → package → class. 5. Add Selenium `.jar` files to project build path.

9. Selenium Imports (Java)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.ie.InternetExplorerDriver;
```

10. Basic WebDriver Syntax

```
WebDriver driver = new ChromeDriver();

driver.get("https://example.com");

String title = driver.getTitle();
String currentUrl = driver.getCurrentUrl();

driver.navigate().refresh();

driver.close();    // closes current window
driver.quit();     // closes all windows
```

11. Web Elements

Common web elements: - Text boxes - Buttons - Links - Radio buttons - Checkboxes - Dropdowns - Auto-suggestion boxes - Images

Locating elements:

```
driver.findElement(By.className("classname")).click();
```

Dynamic attributes may change at runtime → avoid using unstable locators.

12. CSS Selectors

Type	Syntax	Example
ID	<code>#id</code>	<code>#app</code>
Class	<code>.className</code>	<code>.orangehrm-login-button</code>
Attribute	<code>[attr='value']</code>	<code>[name='username']</code>
Tag + Attribute	<code>tag[attr='value']</code>	<code>button[type='submit']</code>
Child	<code>parent > child</code>	<code>div.oxd-form-actions > button</code>

13. Handling Alerts

```
Alert alert = driver.switchTo().alert();

String text = alert.getText();
alert.accept();    // OK
alert.dismiss();  // Cancel
alert.sendKeys("text");
```

14. Wait Mechanisms in Selenium

Selenium waits are used to handle synchronization issues between the test script and the web application. They tell Selenium **how long to wait** before throwing an exception if an element is not found.

Types of Waits

1. Implicit Wait

- Applied globally.
- Selenium waits for a specified time before throwing `NoSuchElementException`.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

2. Explicit Wait (Most Recommended)

- Applied to a specific element.
- Uses conditions like visibility, clickability, etc.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
```

3. Fluent Wait

- Similar to Explicit Wait but more flexible.
- Allows polling frequency and ignoring exceptions.

```
Wait<WebDriver> wait = new FluentWait<>(driver)  
    .withTimeout(Duration.ofSeconds(30))  
    .pollingEvery(Duration.ofSeconds(5))  
    .ignoring(NoSuchElementException.class);
```

Note: Avoid using `Thread.sleep()` as it pauses execution unnecessarily.

14. XPath

Absolute XPath

- Starts from root of HTML
- Very sensitive

Example:

```
/html/body/div/input
```

Relative XPath (Recommended)

Single Attribute

```
//input[@name='username']
```

Multiple Attributes (AND / OR)

```
//input[@type='text' and @name='username']
```

Contains

```
//input[contains(@id,'user')]
```

Starts-With

```
//button[starts-with(text(),'Sign')]
```

15. Drop-Down Handling

```
import org.openqa.selenium.support.ui.Select;

Select dd = new Select(driver.findElement(By.id("id")));

dd.selectByVisibleText("India");
dd.selectByValue("IN");
dd.selectByIndex(1);
```

Other methods: - `isMultiple()` - `deselectAll()` - `getOptions()`

16. Handling IFrames

```
driver.switchTo().frame("frameName");
driver.switchTo().frame(0);
driver.switchTo().frame(element);

driver.switchTo().defaultContent();
```

17. Handling Multiple Windows

```
String parent = driver.getWindowHandle();
Set<String> windows = driver.getWindowHandles();

for(String win : windows){
    if(!win.equals(parent)){
        driver.switchTo().window(win);
    }
}
```

18. Test Frameworks (TestNG / JUnit)

Test frameworks help in **organizing, executing, and reporting** test cases efficiently.

Common Frameworks

- **TestNG** (most widely used with Selenium)
- **JUnit**

Common TestNG Annotations

```
@BeforeMethod    // runs before each test
@AfterMethod     // runs after each test
@Test            // defines a test case
```

Example

```
@Test
public void loginTest() {
    Assert.assertEquals(driver.getTitle(), "Dashboard");
}
```

18. Actions Class (Mouse & Keyboard)

```
Actions actions = new Actions(driver);
```

Mouse actions: - `moveToElement()` - `contextClick()` - `doubleClick()` - `dragAndDrop()`

Keyboard actions: - `keyDown()` - `keyUp()` - `sendKeys()`

Example (Capital typing):

```
actions.moveToElement(input)
    .click()
    .keyDown(Keys.SHIFT)
    .sendKeys("hello")
    .keyUp(Keys.SHIFT)
    .perform();
```

Assertions in Selenium

Assertions are used to **validate expected results** with actual results. If an assertion fails, the test case is marked as failed.

Common Assertions (TestNG)

```
Assert.assertEquals(actual, expected);
Assert.assertTrue(condition);
Assert.assertFalse(condition);
```

Difference: - **Assert** → stops execution on failure - **Verify** → continues execution (mainly in Selenium IDE)

19. Taking Screenshots

```
TakesScreenshot ts = (TakesScreenshot) driver;
File src = ts.getScreenshotAs(OutputType.FILE);
File dest = new File("./screenshots/home.png");
FileUtils.copyFile(src, dest);
```

Page Object Model (POM)

Page Object Model is a **design pattern** used in Selenium to create an object repository for web elements.

Key Concepts

- Each web page is represented as a **separate class**.
- Web elements are defined once and reused.
- Improves **code readability and maintainability**.

Advantages of POM

- Reduces code duplication
- Easy maintenance
- Better test structure

Simple Example

```
public class LoginPage {
    WebDriver driver;
```



```

By username = By.id("username");
By password = By.id("password");
By loginBtn = By.id("login");

public LoginPage(WebDriver driver) {
    this.driver = driver;
}

public void login(String user, String pass) {
    driver.findElement(username).sendKeys(user);
    driver.findElement(password).sendKeys(pass);
    driver.findElement(loginBtn).click();
}
}

```

20. Handling Web Tables

HTML Structure

```

<table>
  <tr>
    <th>First</th>
    <th>Second</th>
  </tr>
  <tr>
    <td>Ragul</td>
    <td>Ravi</td>
  </tr>
</table>

```

Selenium Example

```

List<WebElement> rows = driver.findElements(By.xpath("//table/tr"));

for(int i = 2; i <= rows.size(); i++){
    String firstCol = driver.findElement(By.xpath("//table/tr["+i+"]/td[1]")).getText();
    String secondCol = driver.findElement(By.xpath("//table/tr["+i+"]/td[2]")).getText();
    System.out.println(firstCol + " " + secondCol);
}

```